Introduction to Digital Logic Design Lab
EECS 31L

Lab #03

Student Name: STEVEN KUZHIPALA

Student ID #: 51378962

Date: 2/21/26

D-FF (D Flip Flop)

1. Design & Reasoning:

The objective of this design is to create a synchronous D flip-Flop, with a reset switch. Since the reset switch, and the output value of q must be updated synchronously i.e. along with a rising clock cycle, the module has a always block that is only triggered during the positive edge of the clock.

After which a simple if statement is used to check the reset switch, which if on sets the output q to 0, else updates it with the value from d. The output q is updated using non-blocking statements, to allow the assignments to the variables to happen synchronously along with a rising clock cycle.

```
`timescale 1ns / 1ps

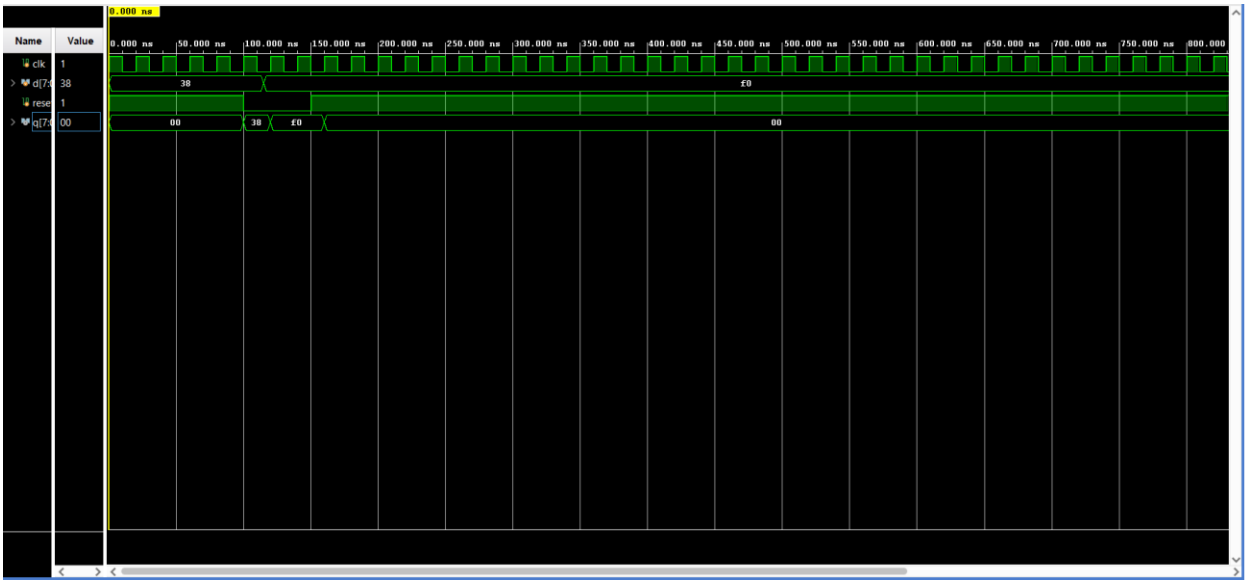
module FlipFlop(clk, reset,d, q);

//Defining input and output signals
    input wire clk;
    input wire reset;
    input wire [7:0]d;
    output reg [7:0] q;
//Defining Flip-Flop behaviour

    always @(posedge clk) begin
        if(!reset)begin
        q<=d;
        end
        else begin
        q <=8'b0;
        end
     end
endmodule
```

2.Testing and Simulation Verification:

```
`timescale 1ns / 1ps
module tb_FlipFlop();
//Define I/O ports
    reg clk;
    reg [7:0] d;
    reg reset;
    wire [7:0] q;
//Port Mapping
FlipFlop instant
(
    .clk(clk),
    .reset(reset),
    .d(d),
    .q(q)
);
//handling clock cycles
initial begin
    clk=1'b1;
    forever #10 clk=~clk;
    end
//handling RESETS
initial
    begin
    //q=7'b0;
    reset=1'b1;
    #100;
    reset=1'b0;
    #50;
    reset=1'b1;
    end
//handling Input d
initial
    begin
    d = 8'h38;
    #115;
    d = 8'hf0;
    end
```



-The reset works synchronously with the clock, turning all the output to 0 at a rising clock cycle, despite the value of the input given.

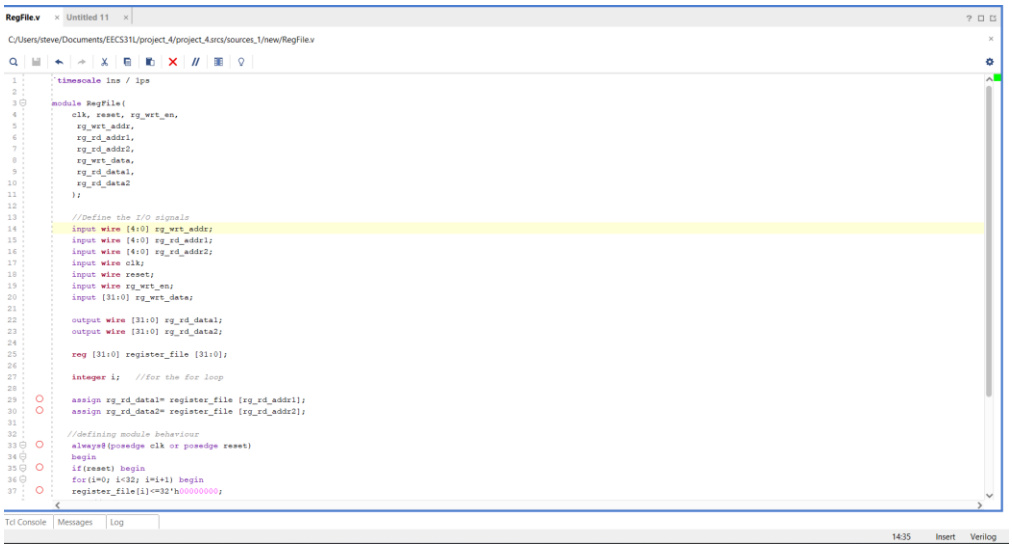
-Data on input line accurately matches the output line, when reset is OFF. Updates only with rising clock cycle.

Register File

1. Design & Reasoning:

The objective of this design is to create a reg based memory, with reading memory designed in such a way that despite the clock/reset values.

The module also has a reset switch that resets all the registers to 0, on its positive edge, and only write new data into registers if the write enable is turned on, and they reset switch is not ON.




```
`timescale 1ns / 1ps

module RegFile(
    clk, reset, rg_wrt_en,
     rg_wrt_addr,
     rg_rd_addr1,
     rg_rd_addr2,
     rg_wrt_data,
     rg_rd_data1,
     rg_rd_data2
    );

    //Define the I/O signals
    input wire [4:0] rg_wrt_addr;
    input wire [4:0] rg_rd_addr1;
    input wire [4:0] rg_rd_addr2;
    input wire clk;
    input wire reset;
    input wire rg_wrt_en;
    input [31:0] rg_wrt_data;

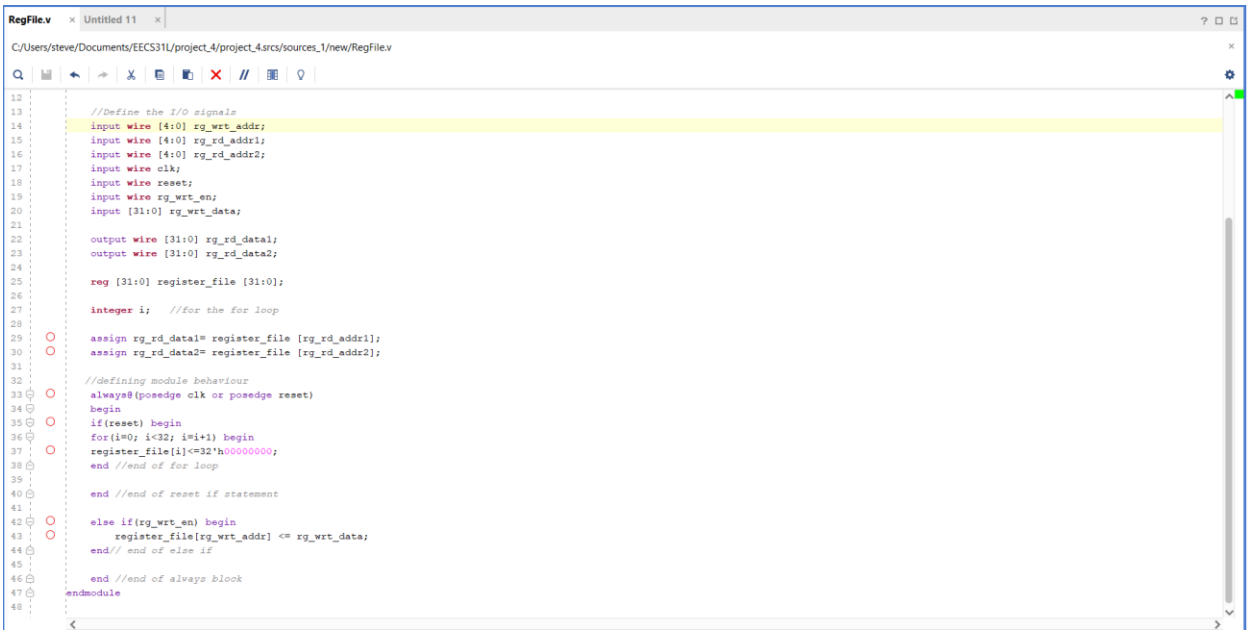
    output wire [31:0] rg_rd_data1;
    output wire [31:0] rg_rd_data2;

    reg [31:0] register_file [31:0];

    integer i;   //for the for loop

    assign rg_rd_data1= register_file [rg_rd_addr1];
    assign rg_rd_data2= register_file [rg_rd_addr2];

    //defining module behaviour
    always@(posedge clk or posedge reset)
    begin
    if(reset) begin
    for(i=0; i<32; i=i+1) begin
    register_file[i]<=32'h00000000;
```




```
    //Define the I/O signals
    input wire [4:0] rg_wrt_addr;
    input wire [4:0] rg_rd_addr1;
    input wire [4:0] rg_rd_addr2;
    input wire clk;
    input wire reset;
    input wire rg_wrt_en;
    input [31:0] rg_wrt_data;

    output wire [31:0] rg_rd_data1;
    output wire [31:0] rg_rd_data2;

    reg [31:0] register_file [31:0];

    integer i;   //for the for loop

    assign rg_rd_data1= register_file [rg_rd_addr1];
    assign rg_rd_data2= register_file [rg_rd_addr2];

    //defining module behaviour
    always@(posedge clk or posedge reset)
    begin
    if(reset) begin
    for(i=0; i<32; i=i+1) begin
    register_file[i]<=32'h00000000;
    end //end of for loop

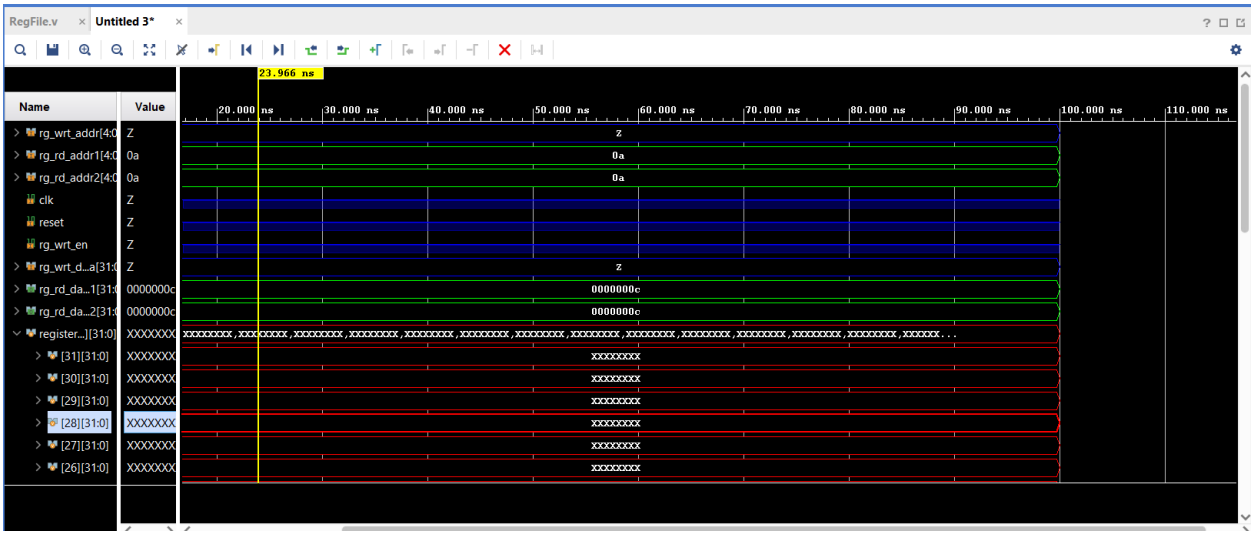
    end //end of reset if statement

    else if(rg_wrt_en) begin
        register_file[rg_wrt_addr] <= rg_wrt_data;
    end// end of else if

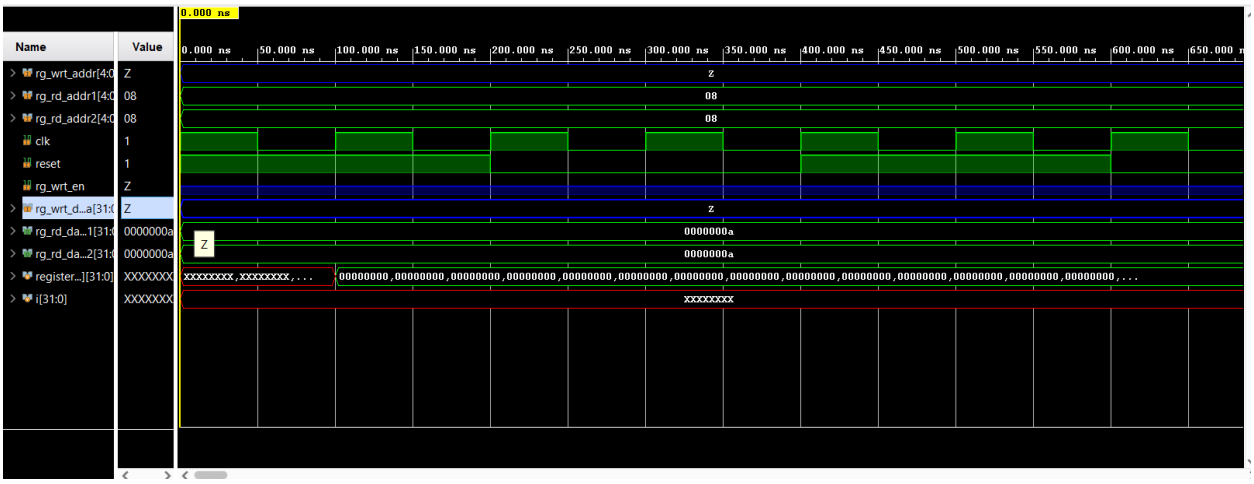
    end //end of always block
endmodule
```

2.Testing and Simulation Verification:

Testing Sample 1:

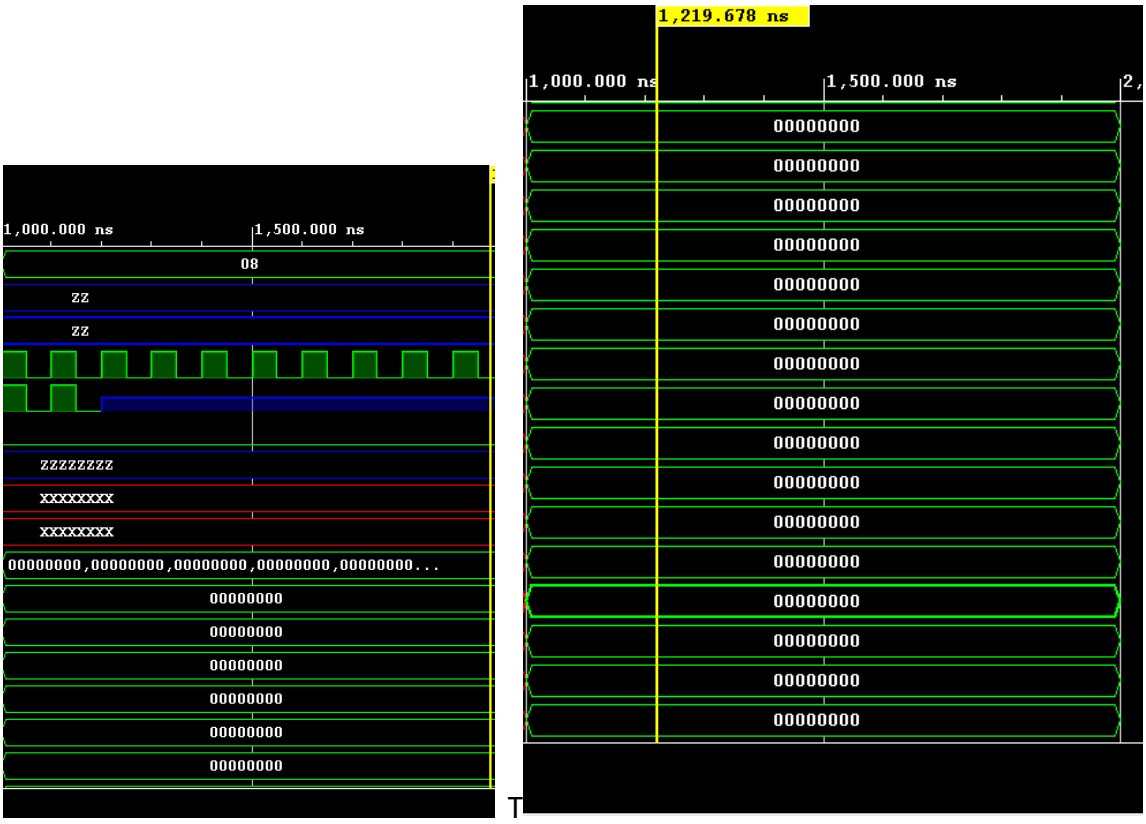


- Regardless of clock/reset, the memory address placed on read memory outputs the data at that particular register.



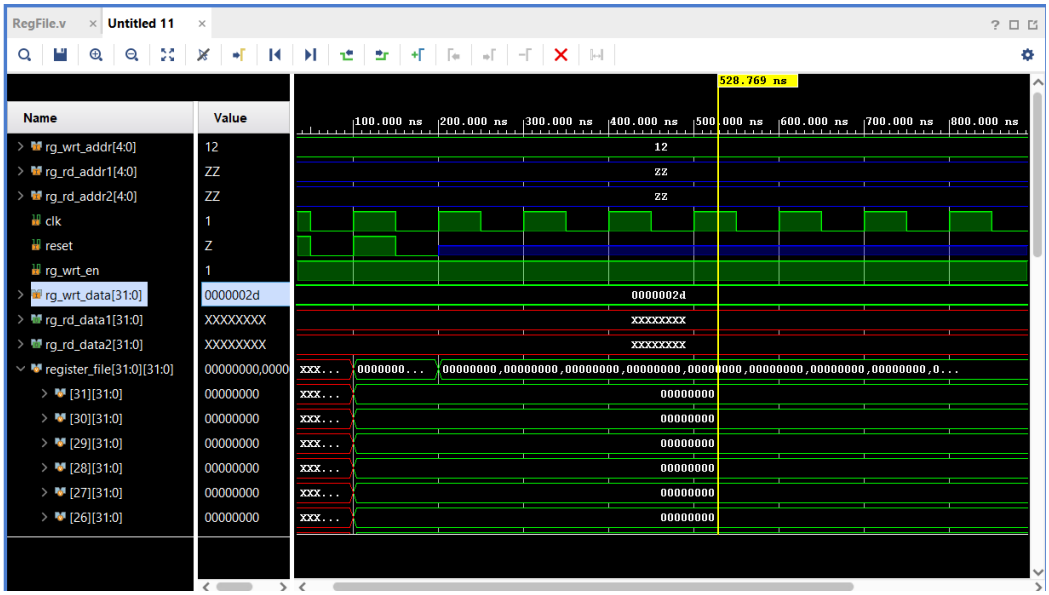
- Right at the rising positive edge of the clock, the reading is done, NOT at the falling edge.

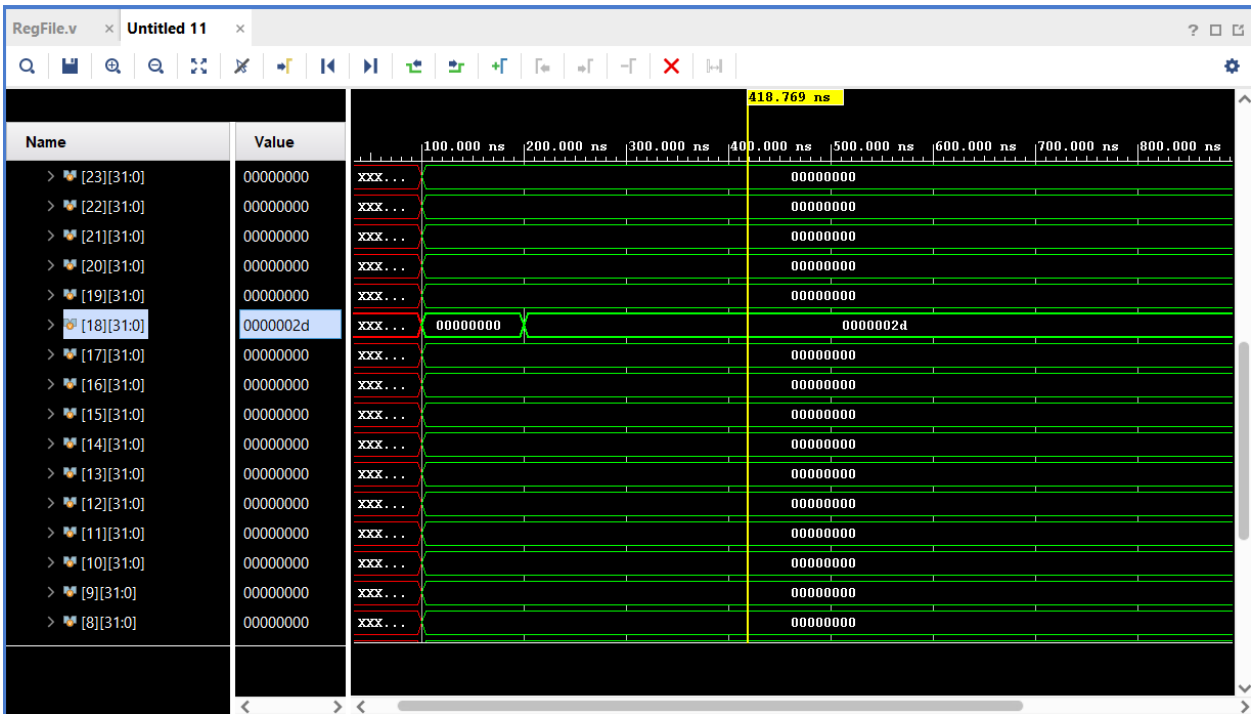
Testing Sample 2:



-No data is written onto registers unless the write enable is turned ON.

Lab Manual Test Sample:

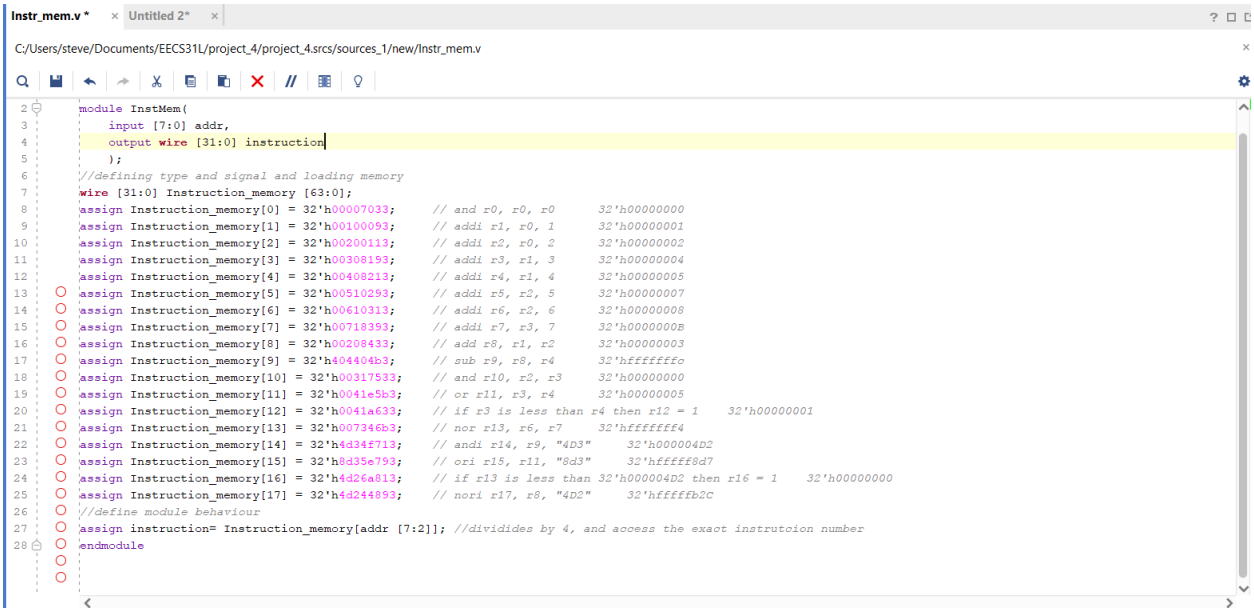



RegFile.v
Untitled 11
Name
Value
[23][31:0]
[22][31:0]
[21][31:0]
[20][31:0]
[19][31:0]
[18][31:0]
[17][31:0]
[16][31:0]
[15][31:0]
[14][31:0]
[13][31:0]
[12][31:0]
[11][31:0]
[10][31:0]
[9][31:0]
[8][31:0]
00000000
0000002d
418.769 ns
100.000 ns
200.000 ns
300.000 ns
400.000 ns
500.000 ns
600.000 ns
700.000 ns
800.000 ns
xxx...

Instruction Memory

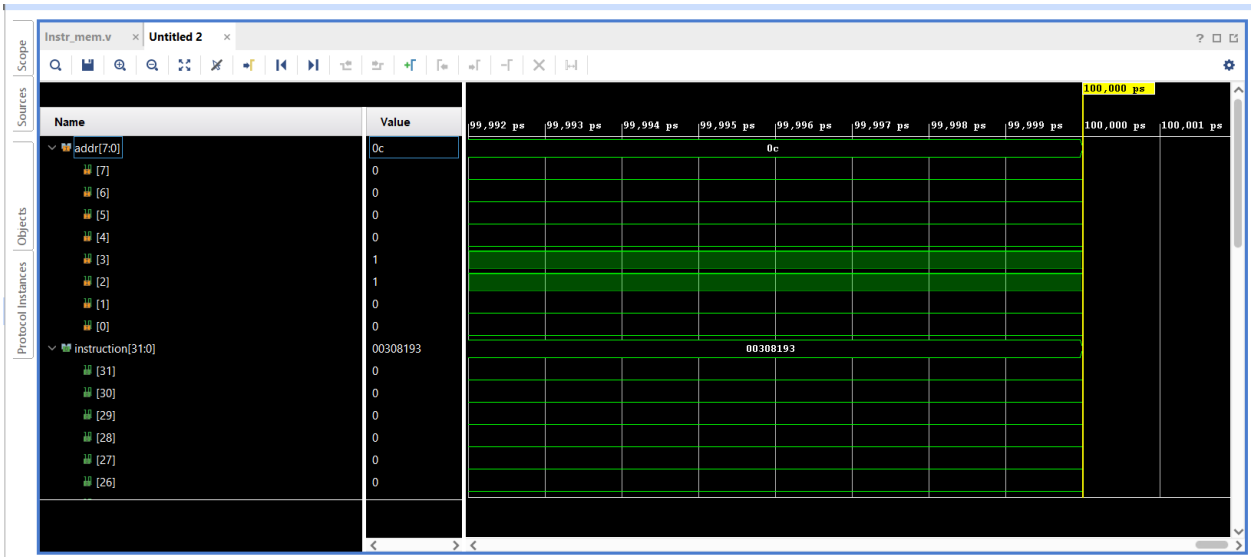
1. Design & Reasoning:

The objective of this module is to design a memory element that can hold up to 64 pieces of memory, that are 32 bits wide. This access is done without a clock and is immediately available using the assign functionality. Since each of the pieces of memory is 4 bytes long, taking the 7:2 bits of the 8-bit input help us access each 32-bit instruction after splitting them all up into 4 equal bytes each.

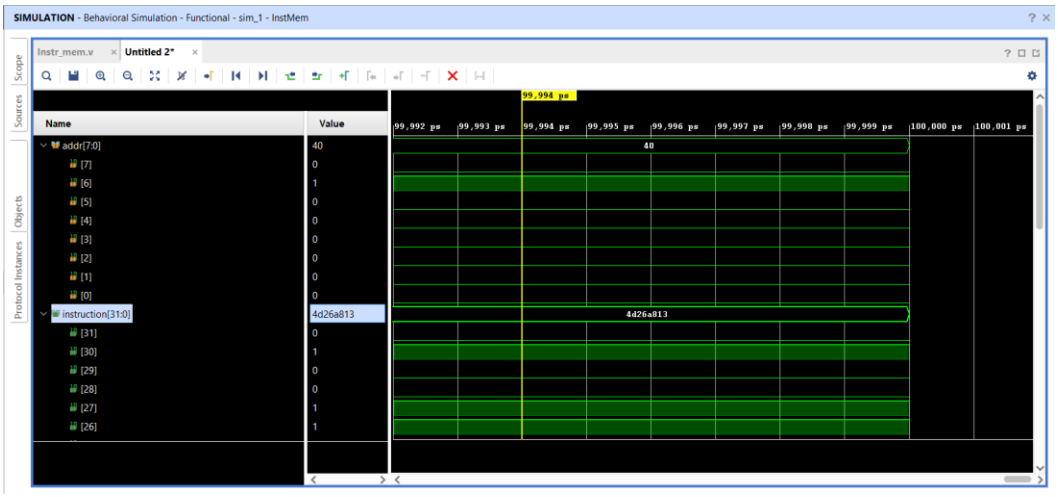


2.Testing and Simulation Verification:

Testing Sample 1:



Testing Sample 2:



Testing Sample 3:

